

Original Paper vs. Proposed Method 1

Original paper

- Spatiotemporal **window-based reconstruction**
- Diffusion state consists of **three consecutive frames**

$$X = (\omega_{t-1}, \omega_t, \omega_{t+1})$$

- Conditional sampling guided by sparse observations and PDE residuals
- **Frames within a window are coupled**
- Each temporal window reconstructed **independently**

Proposed method

- **Sequential filtering** over physical time
- Diffusion used as an **analysis step** within a forecast–analysis loop
- Guide state obtained from **forecasted dynamics**
- Temporal consistency enforced **across windows**

Key change: from window-wise spatiotemporal reconstruction to time-coupled assimilation.

What happens at each physical time step

At time $k \rightarrow k + 1$:

1. Forecast

- Use the dynamics model to move the current estimate forward.
- This gives a *prediction* of the next state.

2. Assimilation

- Compare that prediction with the new observation.
- Adjust it so it:
 - matches the data
 - still respects physics
 - stays on the learned data manifold

3. Output

- The result becomes the best estimate at time $k + 1$.
- This is carried forward to the next step.

Physics-informed Diffusion DA (Overview)

Algorithm: Physics-informed Diffusion Data Assimilation

Require: pretrained ϵ_θ ; forecast $\mathcal{M}$; obs. $H, H^\top$

1: **Require:** data $\{y_k\}_{k=1}^K$; schedule τ, t_d ; weights $\lambda_{\text{phys}}, \lambda_{\text{obs}}$

2: **Require:** steps $\{\gamma_i\}_{i=1}^S$; ensemble size N_e

3: Initialize analysis ensemble $\{s_0^{a,(j)}\}_{j=1}^{N_e}$

4: **for** $k = 0, 1, \dots, K - 1$ **do**

▷ physical time

5: **Forecast:** for $j = 1, \dots, N_e$,

$$\eta_k^{(j)} \sim \mathcal{N}(0, Q), \quad s_{k+1}^{f,(j)} \leftarrow \mathcal{M}(s_k^{a,(j)}) + \eta_k^{(j)}$$

6: **Analysis:** for $j = 1, \dots, N_e$,

$$s_{k+1}^{a,(j)} \leftarrow \text{DIFFUSIONUPDATE}(s_{k+1}^{f,(j)}, y_{k+1})$$

7: **end for**

$$8: \hat{s}_{k+1} \leftarrow \frac{1}{N_e} \sum_{j=1}^{N_e} s_{k+1}^{a,(j)}$$

▷ ensemble mean (reconstruction)

9: $\{\hat{s}_k\}_{k=0}^K$

Physics-informed Diffusion DA (Analysis step)

Subroutine: DIFFUSIONUPDATE($x^{(g)}, y$)

Note: in DA, the guide $x^{(g)}$ is set to the forecast state $s_{k+1}^{f,(j)}$.

Require: guide $x^{(g)}$, observation y , schedule $\tau = \{\tau_0, \dots, \tau_S\}$, noise level t_d

- 1: **Require:** weights $\lambda_{\text{phys}}, \lambda_{\text{obs}}$, steps $\{\gamma_i\}_{i=1}^S$
- 2: $\epsilon \sim \mathcal{N}(0, I)$
- 3: $x_{\tau_S} \leftarrow \sqrt{\bar{\alpha}_{t_d}} x^{(g)} + \sqrt{1 - \bar{\alpha}_{t_d}} \epsilon$
- 4: **for** $i = S, S - 1, \dots, 1$ **do** ▷ diffusion time
- 5: $r_{\tau_i} \leftarrow G|_{u=x_{\tau_i}}; \quad c_{\text{phys}} \leftarrow \partial r_{\tau_i} / \partial x_{\tau_i}$
- 6: $e \leftarrow y - H(x_{\tau_i})$
- 7: $\sigma_{\tau_i}^2 \leftarrow c \cdot \frac{1 - \bar{\alpha}_{\tau_i}}{\bar{\alpha}_{\tau_i}}$ ▷ diffusion noise (SNR-based)
- 8: $R_{\text{eff}} \leftarrow R + \sigma_{\tau_i}^2 I$
- 9: $c_{\text{obs}} \leftarrow H^\top R_{\text{eff}}^{-1} e$ ▷ diffusion-time weighted obs.
- 10: $c_{\text{tot}} \leftarrow \lambda_{\text{phys}} c_{\text{phys}} + \lambda_{\text{obs}} c_{\text{obs}}$
- 11: $\tilde{\epsilon}_\theta \leftarrow \epsilon_\theta(x_{\tau_i}, \tau_i, c_{\text{phys}}) + W[\epsilon_\theta(x_{\tau_i}, \tau_i, c_{\text{phys}}) - \epsilon_\theta(x_{\tau_i}, \tau_i, \emptyset)]$
- 12: $x_{\tau_{i-1}} \leftarrow \text{REVERSESTEP}(x_{\tau_i}, \tilde{\epsilon}_\theta, \tau_i, \tau_{i-1})$
- 13: $x_{\tau_{i-1}} \leftarrow x_{\tau_{i-1}} + \gamma_i c_{\text{tot}}$
- 14: **end for**
- 15: **return** x_{τ_0}

Original Paper vs. Proposed Method 2

Original Paper (Physics-informed DDPM)

- Diffusion model trained with physics guidance
- PDE residuals enter the training objective
- Score network jointly learns data distribution and physics
- Physics tightly coupled to the generative model
- Retraining required for new physical constraints

Proposed Method

- Diffusion model trained *unconditionally* as a data prior
- No physics used during training
- Physics introduced only at inference via likelihood guidance
- Clear role separation:
 - Diffusion: uncertainty modeling and refinement
 - Physics: constraint enforcement
- Modular and transferable across different PDEs

Algorithm: Training

Physics-agnostic DDPM Model Training (Unconditional)

- 1: **Require:** High-fidelity CFD dataset $\mathcal{D}$
- 2: Noise schedule $\{\beta_t\}_{t=1}^T$, define $\alpha_t := 1 - \beta_t$, $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$
- 3: **Ensure:** Pretrained unconditional DDPM prior $\epsilon_\theta(\cdot)$
- 4: **repeat**
- 5: $x_0 \sim q(x_0)$
- 6: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 7: $\epsilon \sim \mathcal{N}(0, \mathbf{I})$
- 8: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
- 9: Update θ by gradient descent on:
- 10: $\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2$
- 11: **until** converged

Algorithm: Inference

Physics-likelihood Guided Conditional Sampling with Pretrained DDPM

```
1:  $x_{(1)}^{(g)} \leftarrow x^{(g)}$ 
2: for  $k = 1, 2, \dots, K$  do
3:   Sample  $\epsilon \sim \mathcal{N}(0, I)$ 
4:    $x_{\tau_S} \leftarrow \sqrt{\bar{\alpha}_{t_d}} x_{(k)}^{(g)} + \sqrt{1 - \bar{\alpha}_{t_d}} \epsilon$  ▷ Noise injection at intermediate diffusion time
5:   for  $i = S, S - 1, \dots, 1$  do
6:      $\hat{\epsilon} \leftarrow \epsilon_{\theta}(x_{\tau_i}, \tau_i)$ 
7:      $\hat{x}_0 \leftarrow \frac{x_{\tau_i} - \sqrt{1 - \bar{\alpha}_{\tau_i}} \hat{\epsilon}}{\sqrt{\bar{\alpha}_{\tau_i}}}$  ▷ Denoised 3-frame estimate
8:      $r \leftarrow G(\hat{x}_0)$  ▷ PDE residual on window
9:      $g_{\text{phys}} \leftarrow \nabla_{\hat{x}_0} \left( \frac{1}{2} \|r\|^2 \right)$  ▷ Physics likelihood gradient
10:     $\tilde{\epsilon} \leftarrow \hat{\epsilon} - \lambda_{\text{phys}} \sqrt{1 - \bar{\alpha}_{\tau_i}} g_{\text{phys}}$ 
11:     $x_{\tau_{i-1}} \leftarrow \text{REVERSESTEP}(x_{\tau_i}, \tilde{\epsilon}, \tau_i, \tau_{i-1})$ 
12:  end for
13:   $x_{(k+1)}^{(g)} \leftarrow x_{\tau_0}$  ▷ Recursive refinement (optional)
14: end for
15: return  $x_{\tau_0}$ 
```

Original method vs Proposed method 3

Original method

- Diffusion trained directly in pixel (flow-field) space
- Sampling initialized from noised guide $x^{(g)}$
- Diffusion jointly learns:
 - flow manifold
 - reconstruction
 - uncertainty
- Requires many reverse steps under extreme sparsity
- Can be unstable with very limited observations

Proposed method

- Self-supervised pixel-level pretraining to learn flow structure
- Flow manifold learned *before* diffusion
- Diffusion initialized from a physically plausible state
- Clear role separation:
 - Pretraining: structure and inductive bias
 - Diffusion: uncertainty modeling and refinement
- Fewer diffusion steps with improved stability

Algorithm: Training Spatiotemporal MAE

Require: Clips $(\omega_{t-1}, \omega_t, \omega_{t+1})$, $M \sim \mathcal{M}$, MAE (E_ϕ, D_ϕ)

Require: $\lambda_{\text{mid}} > 0$, optional $\lambda_{\text{spec}} \geq 0$, $\delta > 0$

```
1: for each minibatch do
2:    $\tilde{\omega}_t \leftarrow M \odot \omega_t + (1 - M) \odot 0$ 
3:    $\hat{\omega}_t \leftarrow D_\phi(E_\phi(\omega_{t-1}, \tilde{\omega}_t, \omega_{t+1}, M))$ 
4:    $\mathcal{L}_{\text{rec}} \leftarrow \|(1 - M) \odot (\hat{\omega}_t - \omega_t)\|_2^2 + \lambda_{\text{mid}} \|M \odot (\hat{\omega}_t - \omega_t)\|_2^2$ 
5:   if  $\lambda_{\text{spec}} > 0$  then
6:      $\mathcal{L}_{\text{spec}} \leftarrow \|\log(E_{\hat{\omega}}(k) + \delta) - \log(E_{\omega}(k) + \delta)\|_2^2$ 
7:   else
8:      $\mathcal{L}_{\text{spec}} \leftarrow 0$ 
9:   end if
10:   $\mathcal{L} \leftarrow \mathcal{L}_{\text{rec}} + \lambda_{\text{spec}} \mathcal{L}_{\text{spec}}$ 
11:   $\phi \leftarrow \phi - \eta \nabla_\phi \mathcal{L}$ 
12: end for
```

Algorithm: Conditional DDPM Training (3-frame state)

Conditional DDPM Training for Spatiotemporal Vorticity Reconstruction

Require: Training clips $x_0 = (\omega_{t-1}, \omega_t, \omega_{t+1}) \sim q(x_0)$

Require: Diffusion model ϵ_θ , steps S , schedule $\{\bar{\alpha}_s\}_{s=1}^S$, learning rate η

Require: Mask distribution $\mathcal{M}$ (mixture of mask types):

$\mathcal{M} = \text{Mix}(\mathcal{M}_{\text{block}}, \mathcal{M}_{\text{point}}, \mathcal{M}_{\text{lattice}})$

$\mathcal{M}_{\text{block}}$: random block masking (MAE-style)

$\mathcal{M}_{\text{point}}$: random point/collocation masks (sparse sensors)

$\mathcal{M}_{\text{lattice}}$: structured lattice masks (low-res grids; stride 4/8)

- 1: **for** each minibatch **do**
- 2: Sample clean state $x_0 = (\omega_{t-1}, \omega_t, \omega_{t+1})$
- 3: Sample mask $M \sim \mathcal{M}$ and set observations $y \leftarrow M \odot \omega_t$
- 4: Sample timestep $s \sim \text{Uniform}(\{1, \dots, S\})$
- 5: Sample noise $\varepsilon \sim \mathcal{N}(0, I)$
- 6: Form noisy state $x_s \leftarrow \sqrt{\bar{\alpha}_s} x_0 + \sqrt{1 - \bar{\alpha}_s} \varepsilon$
- 7: Define conditioning $c \leftarrow (y, M)$
- 8: Predict noise $\hat{\varepsilon} \leftarrow \epsilon_\theta(x_s, s; c)$

▷ ε has same shape as x_0

Noise-prediction loss and explicit parameter update

- 9: $\mathcal{L}_{\text{diff}}(\theta) \leftarrow \|\hat{\varepsilon} - \varepsilon\|_2^2$
- 10: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{diff}}(\theta)$
- 11: **end for**

Algorithm: Inference

Spatiotemporal Masked Autoencoder + Diffusion

Require: Sparse observations y , binary mask $M \in \{0, 1\}^{H \times W}$

Require: Neighboring frames (x_{t-1}, x_{t+1})

Require: MAE encoder-decoder (E_ϕ, D_ϕ) , diffusion model ϵ_θ

Require: Steps S , noise schedule $\{\sigma_s\}_{s=1}^S$

Require: Target spectrum $E^*(k)$, stabilizer $\delta > 0$

1: $\tilde{x}_t \leftarrow M \odot y + (1 - M) \odot 0$

2: $\hat{x}_t^{(0)} \leftarrow D_\phi(E_\phi(x_{t-1}, \tilde{x}_t, x_{t+1}, M))$

3: $c \leftarrow (y, M)$

4: Sample $\varepsilon \sim \mathcal{N}(0, I)$

5: $x_S \leftarrow \hat{x}_t^{(0)} + \sigma_S \varepsilon$

6: **for** $s = S, S-1, \dots, 1$ **do**

7: $\hat{\varepsilon} \leftarrow \epsilon_\theta(x_s, s; c)$

8: $x_{s-1} \leftarrow \text{REVERSESTEP}(x_s, \hat{\varepsilon}, s)$

9: $x_{s-1} \leftarrow M \odot y + (1 - M) \odot x_{s-1}$

10: $\hat{x} \leftarrow \mathcal{F}(x_{s-1})$

11: $E(k) \leftarrow \text{RADIALPOWERSPECTRUM}(\hat{x})$

▷ FFT

12: $g(k) \leftarrow \sqrt{\frac{E^*(k)}{E(k) + \delta}}$

13: $\hat{x} \leftarrow g(k) \odot \hat{x}$

14: $x_{s-1} \leftarrow \mathcal{F}^{-1}(\hat{x})$

▷ IFFT

15: **end for**

16: **return** $\hat{x}_t \leftarrow x_0$

Radial Power Spectrum (Vorticity). Let $\hat{\omega}(k_x, k_y) = \mathcal{F}(\omega)$ denote the 2D Fourier transform of the vorticity field. The radial (enstrophy) spectrum is defined as

$$E_{\omega}(k) = \sum_{\substack{(k_x, k_y): \\ k \leq \sqrt{k_x^2 + k_y^2} < k + \Delta k}} |\hat{\omega}(k_x, k_y)|^2,$$

where $k = \sqrt{k_x^2 + k_y^2}$ is the wave number and Δk denotes the radial bin width.